



École Nationale Supérieure d'Informatique
pour l'Industrie et l'Entreprise

Rapport de stage de première année

Conception d'un tableau de bord interne

ÉTUDIANT

Omar DAHMAN

TUTEUR ACADÉMIQUE

David ROUSSEL

MAÎTRE DE STAGE

Louail POTHIN

ENTREPRISE

RT MOVE



RT MOVE — 26 Rue de la Frette, 78500 Sartrouville

1^{er} juillet — 28 août 2026

Remerciements

Je remercie mon maître de stage, Louail Pothin, pour son accueil, sa disponibilité et ses conseils pendant tout le stage. Je remercie aussi le gérant de RT MOVE de m'avoir permis de faire ce stage dans son entreprise et de m'avoir fait confiance sur ce projet. Merci également à mon tuteur académique, David Roussel, pour son suivi pendant ces deux mois. Enfin, je remercie Aurélien, Mounib, Israa et David, les quatre autres stagiaires, pour la bonne entente et l'entraide qui ont rendu ce travail d'équipe agréable et efficace.

Résumé

Ce stage, fait chez RT MOVE (entreprise de déménagement) du 1^{er} juillet au 28 août 2026, fait partie d'un projet de digitalisation mené par une équipe de cinq stagiaires. Ma mission (lot L5) a consisté à construire un **tableau de bord web interne** pour suivre l'activité en direct : carte des camions et des missions, calcul d'itinéraires, suivi en direct des positions, indicateurs commerciaux et marketing, et alertes.

Le tableau de bord a été construit avec Next.js/React/TypeScript, avec Leaflet et OpenRouteService pour la carte, et Supabase pour les données. J'ai fait attention à ce que le tableau de bord **marche même si les autres lots ne sont pas terminés** : chaque flux en direct repasse automatiquement en vérification périodique si la connexion en direct ne marche pas, et le lien avec la base de données partagée a été vérifiée par des tests directs sur l'API.

En plus du tableau de bord, j'ai aussi construit un **assistant de discussion** (chatbot) pour le site public de RT MOVE, proposé par mon maître de stage : il répond aux questions des visiteurs et enregistre les personnes intéressées comme de vrais prospects, directement dans la base de données.

Lors de la dernière semaine du stage, des tests ont été menés sur l'ensemble des outils construits, et une démonstration a été faite au gérant de RT MOVE pour lui expliquer comment utiliser au quotidien le tableau de bord et le chatbot.

Glossaire

Next.js / React Outils JavaScript/TypeScript pour construire des sites et applications web.

Supabase Plateforme basée sur PostgreSQL qui fournit une base de données, un système de connexion, du stockage de fichiers et des mises à jour en direct.

RLS (Row Level Security) Règle de PostgreSQL qui limite l'accès aux lignes d'une table selon la personne connectée.

Realtime Canal de Supabase qui envoie en direct les changements de la base (`postgres_changes`) au navigateur, sans recharger la page.

@supabase/ssr Outil officiel de Supabase qui permet de partager la même connexion (via les cookies) entre le navigateur et le serveur.

Polling Vérification périodique de la base, utilisée ici comme solution de secours quand Realtime ne marche pas.

Leaflet / OpenStreetMap (OSM) Outil pour afficher des cartes, avec un fond de carte libre et gratuit.

OpenRouteService (ORS) Service qui calcule un itinéraire et un temps de trajet estimé (ETA).

KPI *Key Performance Indicator*, indicateur clé pour suivre les performances (commercial, marketing).

Gemini Modèle de langage (intelligence artificielle) de Google, utilisé pour l'assistant de discussion du site public.

Appel d'outil (*function calling*) Fonctionnement qui permet à un modèle de langage de déclencher une action précise (ici, enregistrer un prospect) plutôt que de simplement répondre par du texte.

Réponse en flux (*streaming*) Mode d'affichage où la réponse du modèle apparaît mot par mot au fur et à mesure de sa génération, plutôt que d'attendre la réponse complète avant de l'afficher.

ADR *Architecture Decision Record*, document qui explique un choix technique et pourquoi il a été fait.

PR (Pull Request) Demande pour fusionner une branche Git dans la branche principale, relue par quelqu'un d'autre avant d'être acceptée.

Jalon (J1–J6) Point de contrôle commun à toute l'équipe, aux dates importantes du stage.

Table des figures

3.1	Page de connexion de l'espace interne	5
3.2	Les quatre pages du tableau de bord	6
3.3	L'assistant de discussion sur le site public, après l'enregistrement réussi d'un prospect	7

Table des matières

Remerciements	i
Résumé	ii
Glossaire	iii
Table des figures	iv
1 Introduction : l'entreprise et le stage	1
1.1 L'entreprise RT MOVE	1
1.2 Contexte du projet de digitalisation	1
1.3 Une équipe de cinq stagiaires	1
1.4 Ma mission (lot L5) et ses objectifs	2
2 Contexte et notions utiles	3
2.1 Outils utilisés	3
2.2 Notions clés	3
2.2.1 Cartes web et itinéraires	3
2.2.2 En direct : Realtime contre polling	3
2.2.3 Sécurité des données (RLS)	3
2.3 Mise en place de l'environnement	3
3 Ce qui a été réalisé	5
3.1 Architecture générale	5
3.2 Carte et itinéraires	5
3.3 Suivi en direct et fiabilité	5
3.4 Mise à jour automatique	6
3.5 Alertes de pilotage	6
3.6 Indicateurs métier (KPI)	7
3.7 Un assistant de discussion pour le site public	7
3.7.1 Fonctionnement	7
3.7.2 Choix technique	8
3.7.3 Vérifications faites et corrections apportées	8
3.7.4 Organisation et statut	8
3.8 Intégration à la base Supabase	9
3.8.1 Principe et règle de sécurité	9
3.8.2 Un blocage trouvé, puis résolu	9
3.8.3 Passage à la vraie connexion	9

3.8.4	Pourquoi certaines tables restaient vides, et ce qui a changé depuis	9
3.9	Intégration avec la base du CRM	10
3.10	Refonte du design	10
3.11	Démarche de vérification	10
3.12	Difficultés rencontrées	10
3.12.1	Un cache de compilation abîmé	10
3.12.2	Des droits de base non ouverts au départ	11
3.12.3	Dépendances entre les lots	11
3.12.4	Une fausse alerte sur les données marketing	11
3.12.5	Du texte généré sans accents	11
4	Conclusion	12
A	DD&RS — Développement durable et responsabilité sociétale	13
B	Détails techniques	14
B.1	Outil de suivi en direct avec repli polling	14
B.2	Alertes de pilotage	14
B.3	Appel sécurisé à OpenRouteService (côté serveur)	14
B.4	Client Supabase côté navigateur (vraie connexion)	15
B.5	Assistant de discussion : réponse en streaming avec appel de l'outil <code>save_lead</code>	15
	Bibliographie et webographie	15

Chapitre 1

Introduction : l'entreprise et le stage

1.1 L'entreprise RT MOVE

RT MOVE est une entreprise de transport, de déménagement et de nettoyage. Elle intervient en Île-de-France et dans les régions limitrophes. Pour se développer, l'entreprise veut améliorer ses outils numériques afin de mieux gérer son activité, rassembler ses données au même endroit, et automatiser la recherche de clients professionnels : c'est le but du projet de digitalisation décrit dans ce rapport.

L'entreprise est dirigée par deux personnes, inscrites au Registre National des Entreprises (RNE) tenu par l'INPI :

- **Louail Giovanni POTHIN**, directeur général — également mon maître de stage ;
- **Thomas ROLLOT**, président de la société.

1.2 Contexte du projet de digitalisation

Avant ce projet, l'activité commerciale et l'organisation des missions étaient suivies avec des fichiers Excel (prospection, ventes, suivi des missions, contenus marketing). Cette façon de faire montrait ses limites : saisie à la main, aucune vue en direct, et difficulté à rassembler les chiffres. Le projet a pour but de remplacer petit à petit ces fichiers par un vrai système (base de données, CRM, application mobile pour le terrain, automatisations et tableau de bord).

1.3 Une équipe de cinq stagiaires

Le projet est découpé en lots (L0 à L8) partagés entre cinq stagiaires :

- **David** — Base technique et automatisation, prospection et marketing automatique (L0, L6).
- **Aurélien** — Chef de produit et données, modélisation de la base, partie visible du CRM, recette (L1, L3, L8).
- **Mounib** — Backend et sécurité, base de données, API, sécurité (L1, L2, L4).
- **Israa** — Application mobile pour le terrain, suivi GPS (L7) — *a quitté le stage en cours de route.*
- **Omar (moi)** — Tableau de bord web en direct (L5).

Israa a abandonné le stage avant la fin du lot L7. Sa partie (application mobile, remontée des positions GPS) n'a donc pas été livrée. Cela ne remet pas en cause le travail fait sur le lot L5 : la carte et le suivi en direct restent fonctionnels et conçus pour tolérer une table `positions_vehicules` vide (voir section 3.8), mais ils n'afficheront pas de vraies positions tant qu'une autre solution n'aura

pas été trouvée pour ce lot. À la suite de ce départ, mon périmètre s'est recentré sur les indicateurs KPI (commerciaux et marketing), et mon maître de stage m'a proposé un projet complémentaire, l'assistant de discussion du site public (voir section 3.7).

1.4 Ma mission (lot L5) et ses objectifs

Mon lot consiste à construire le tableau de bord interne :

1. une **carte** des camions et des missions du jour (Leaflet + OSM), avec les itinéraires et les temps de trajet estimés (OpenRouteService) ;
2. un **suivi en direct** des positions GPS envoyées par l'application mobile ;
3. des **indicateurs métier** (entonnoir commercial, chiffres marketing) ;
4. une vue de **pilotage du jour** avec des alertes (lead chaud, mission en retard).

Chapitre 2

Contexte et notions utiles

2.1 Outils utilisés

Le tableau de bord utilise : Next.js 16, React 19, TypeScript, Tailwind CSS 4 et shadcn/ui pour l'interface ; Leaflet et OpenStreetMap pour la carte ; OpenRouteService pour les itinéraires ; Recharts pour les graphiques ; et le client JavaScript Supabase (connexion, Realtime, RLS) pour les données. Le chatbot ajoute l'API Gemini (Google) pour la génération des réponses. Le tout se trouve dans le même dépôt que le CRM, dont je réutilise la base commune.

2.2 Notions clés

2.2.1 Cartes web et itinéraires

Leaflet affiche un fond de carte OpenStreetMap et des points personnalisés (camions, départ/arrivée des missions). À partir de coordonnées GPS, OpenRouteService calcule un itinéraire routier (une ligne sur la carte) et un temps de trajet estimé.

2.2.2 En direct : Realtime contre polling

Deux façons permettent de mettre à jour la carte sans recharger la page : un canal qui « pousse » les changements (Supabase Realtime, qui prévient quand la base change) ou une vérification périodique (*polling*). Le choix, expliqué dans un document de décision technique (annexe **B**), retient Supabase Realtime comme solution principale, avec repli automatique en polling.

2.2.3 Sécurité des données (RLS)

Les tables sont protégées par des règles RLS (*Row Level Security*) : seules les requêtes envoyées par une personne connectée (rôle **authenticated**) peuvent lire les données. Le tableau de bord doit donc être connecté pour afficher la moindre ligne (voir section 3.8). Le chatbot du site public fait exception : un visiteur anonyme n'étant pas connecté, l'écriture d'un prospect passe par une connexion Supabase privée, réservée au serveur (voir section 3.7).

2.3 Mise en place de l'environnement

L'environnement de développement (dépôt, règles Git, projet Supabase, intégration continue) a été mis en place par David (lot L0). Une fois le projet récupéré, l'installation en local et le lancement

du serveur de développement se font en quelques minutes grâce au guide de démarrage partagé par l'équipe.

Chapitre 3

Ce qui a été réalisé

3.1 Architecture générale

Le tableau de bord est organisé en quatre pages sous `/admin/dashboard` : *pilotage du jour*, *carte en direct*, *KPI commerciaux* et *KPI marketing*. Il s'ajoute par-dessus la base commune du CRM (menu, contrôle de connexion) construite par Aurélien, grâce à une mise en page imbriquée qui ajoute son propre sous-menu. La connexion se fait sur une page dédiée (`/connexion`, voir figure 3.1), dont la session est ensuite partagée par tout l'espace interne.



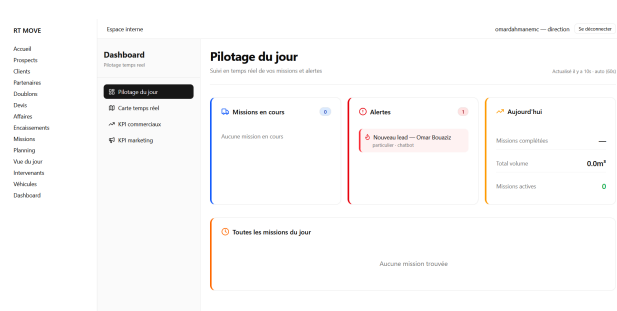
FIGURE 3.1 – Page de connexion de l'espace interne

3.2 Carte et itinéraires

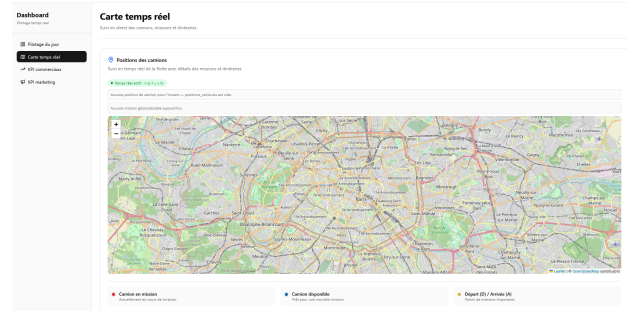
La carte affiche les camions et les missions du jour. Les adresses sont transformées en coordonnées GPS (géocodage), puis OpenRouteService calcule l'itinéraire et le temps de trajet estimé. Les appels à ORS passent par une route serveur Next.js qui protège la clé d'API et gère les erreurs (quota dépassé : code 429, service en panne : code 502).

3.3 Suivi en direct et fiabilité

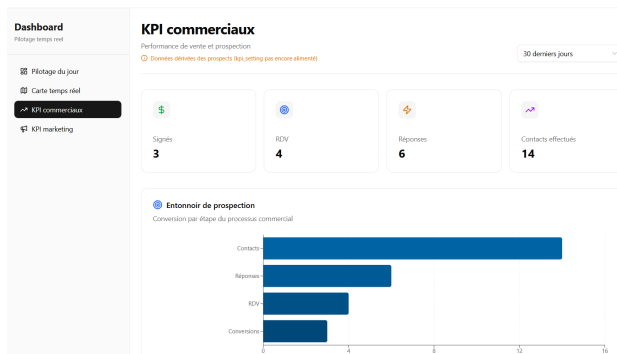
Les positions des véhicules sont lues dans la table `positions_vehicules` et mises à jour en direct grâce à Realtime. Un point important de la conception est la **fiabilité** : si le canal Realtime n'est pas disponible (panne réseau, etc.), l'interface repasse automatiquement en polling, et un badge indique



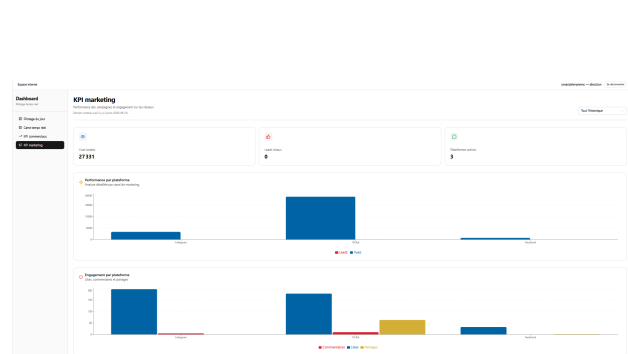
(a) Pilotage du jour



(b) Carte en direct



(c) KPI commerciaux



(d) KPI marketing

FIGURE 3.2 – Les quatre pages du tableau de bord

l'état de la connexion (« Temps réel actif » / « Mode dégradé ») ainsi que l'ancienneté de la dernière mise à jour.

3.4 Mise à jour automatique

Trois pages ne se mettaient à jour qu'au chargement de la page : *pilotage du jour*, *KPI commerciaux* et *KPI marketing*. Elles ont maintenant une **mise à jour automatique toutes les 60 secondes**, grâce à un outil partagé, avec un indicateur du temps écoulé depuis la dernière mise à jour — le même principe que le badge déjà utilisé pour la carte en direct.

3.5 Alertes de pilotage

- La vue de pilotage affiche deux types d'alertes, calculées sur les vraies données (voir figure 3.2a) :
- **Lead chaud** — tout nouveau prospect ajouté dans la table **prospects**, quelle que soit sa source (formulaire de contact, calculateur, Leboncoin, ou désormais le chatbot du site public). L'outil de scoring de David ne garde que les prospects au-dessus d'un certain score ; un nouveau prospect correspond donc à un lead à signaler. L'abonnement Realtime sur les ajouts affiche l'alerte immédiatement.
 - **Mission en retard** — mission dont la date prévue est passée alors que son statut n'est ni « terminée » ni « annulée ».

3.6 Indicateurs métier (KPI)

Trois familles d'indicateurs sont proposées : un *entonnoir commercial* (contacts → réponses → RDV → ventes, depuis `kpi_setting`) et des *indicateurs marketing* (vues et leads par plateforme, depuis `kpi_marketing`). Les graphiques sont faits avec Recharts (voir les pages « KPI commerciaux » et « KPI marketing » en figures 3.2c et 3.2d).

Une **sélection de période** (7 jours, 30 jours, 3 mois ou tout l'historique) a été ajoutée sur ces deux pages, avec un outil générique partagé entre les deux. Cet ajout a permis de trouver un **bug caché** : le calcul de l'entonnoir commercial ne prenait en compte que la toute dernière ligne de KPI enregistrée, au lieu d'additionner toutes les lignes de la période choisie ; il a été corrigé à cette occasion.

Un deuxième graphique, pour l'**engagement** (likes, commentaires, partages par plateforme), a aussi été ajouté sur la page KPI marketing. Il a été séparé du graphique vues/leads existant pour ne pas mélanger des valeurs de tailles très différentes sur un même axe, avec une palette de couleurs vérifiée pour rester lisible par les personnes daltoniennes, grâce à l'outil de vérification du projet.

Un indicateur de fraîcheur (« Dernier contenu suivi il y a N jours », visible en figure 3.2d) a d'abord été conçu pour ne s'afficher qu'en cas de retard (plus de 3 jours sans nouveau contenu). À l'usage, il s'est avéré plus utile de l'afficher **en permanence** : l'affichage a donc été changé pour toujours montrer la date du dernier contenu suivi, retard ou non.

3.7 Un assistant de discussion pour le site public

En plus du tableau de bord interne, mon maître de stage m'a proposé de construire un **assistant de discussion** (chatbot) pour le site public de RT MOVE. Le but est double : répondre aux questions courantes des visiteurs (formules, prix, paiement, assurance, zone d'intervention...) et récupérer les coordonnées des personnes intéressées, pour en faire de vrais prospects.



FIGURE 3.3 – L'assistant de discussion sur le site public, après l'enregistrement réussi d'un prospect

3.7.1 Fonctionnement

L'assistant est un petit bouton de discussion affiché sur toutes les pages du site public. Chaque message envoyé par un visiteur est transmis à un modèle de langage (*Gemini*, de Google), accompagné d'un texte de préparation qui contient les vraies informations du site : les trois formules et leurs tarifs, les conditions de paiement, les règles d'assurance, la zone d'intervention (Île-de-France et régions limitrophes), les coordonnées de l'entreprise et le nom des fondateurs. Ce texte de préparation sert de garde-fou : l'assistant ne doit répondre qu'avec des informations réelles, jamais inventées. Les réponses

s'affichent en flux (*streaming*), mot par mot, plutôt que d'attendre la réponse complète : le ressenti de rapidité est bien meilleur, même si le temps total de réponse ne change pas.

Quand un visiteur donne son nom et un moyen de contact (téléphone ou e-mail), l'assistant peut appeler un **outil** qui enregistre ces informations directement dans la vraie table **prospects**, la même table que celle utilisée par les autres sources de prospects (site Leboncoin, formulaire de contact, calculateur de devis) — voir figure 3.2a, où un lead capturé par le chatbot apparaît dans les alertes du jour au même titre que les autres sources. Un contrôle évite de créer un doublon si la personne existe déjà dans la table ; si elle existe déjà, ses informations sont mises à jour plutôt que dupliquées. Comme un visiteur du site public n'est pas connecté, cette écriture passe par une connexion Supabase privée, réservée au serveur, qui ne sert qu'à cette tâche précise.

3.7.2 Choix technique

Le modèle *Claude* (Anthropic) a d'abord été utilisé pour construire l'assistant, mais son usage est payant dès la première requête. Le projet a donc basculé vers *Gemini* (Google), qui propose un usage gratuit suffisant pour les tests. Un premier essai avec un modèle Gemini récent a montré deux problèmes : un temps de réponse très long (environ 20 secondes) à cause d'un mode de raisonnement interne activé par défaut, et une limite de seulement 20 requêtes gratuites par jour. Le choix s'est donc porté sur un modèle plus léger, qui répond en 2 à 3 secondes avec une limite gratuite bien plus large.

3.7.3 Vérifications faites et corrections apportées

Le fonctionnement a été vérifié directement en base de données, et pas seulement en regardant l'écran : une conversation de test a été menée jusqu'au bout, puis la ligne créée dans **prospects** a été retrouvée par une requête directe. Ces tests ont fait remonter deux problèmes, corrigés ensuite :

- l'appel de l'outil d'enregistrement n'était pas toujours déclenché par le modèle, même quand les conditions étaient réunies (nom et contact fournis) ; la consigne donnée au modèle a été rendue plus stricte et explicite, avec un exemple concret, ce qui a résolu le problème dans les tests répétés ensuite ;
- quand un visiteur corrigeait une information déjà donnée (ex. un numéro de téléphone), l'assistant affirmait avoir mis à jour ses coordonnées alors que la base n'était pas réellement modifiée ; la fonction d'enregistrement a été corrigée pour vraiment mettre à jour la ligne existante dans ce cas, au lieu de ne rien faire.

Une dernière vérification a porté sur l'orthographe : le texte de préparation avait été rédigé sans accents, et le modèle reproduisait cette orthographe fautive dans ses réponses aux visiteurs. Le texte a été réécrit avec les accents corrects, ce qui a corrigé le problème (voir section 3.12.5).

3.7.4 Organisation et statut

Le chatbot étant une fonctionnalité indépendante du tableau de bord (site public contre espace interne), son code a été séparé dans sa propre branche Git (**feat/chatbot-site-public**) et sa propre *Pull Request*, plutôt que d'être mélangé avec celle du lot L5.

L'assistant a été validé par mon maître de stage et mis en ligne sur le site public de RT MOVE lors de la dernière semaine du stage.

3.8 Intégration à la base Supabase

3.8.1 Principe et règle de sécurité

Chaque page interroge directement les vraies tables Supabase grâce à des outils dédiés (`useMissions`, `useKPICommercial`, `useKPIMarketing`, `useVehiclePositions`, `useAlertes`). Les tables étant protégées par RLS pour le seul rôle `authenticated`, le tableau de bord doit s'appuyer sur une connexion valide pour lire la moindre ligne.

3.8.2 Un blocage trouvé, puis résolu

Lors des premiers tests, la connexion échouait avec une erreur de permission : les requêtes partaient de façon anonyme, or la RLS n'autorise que les personnes connectées. En interrogeant **directement l'API de Supabase** (sans passer par l'application), j'ai confirmé que ce n'était pas un bug dans mon code mais un vrai besoin de connexion. En attendant la vraie connexion de Mounib, un compte de service et un contrôle d'accès temporaire (`DashboardAuthGate`) ont été mis en place côté navigateur pour continuer à avancer.

3.8.3 Passage à la vraie connexion

Une fois la vraie connexion Supabase et la migration de la base livrées par Mounib, ainsi que les modules Devis, Missions et Prospects du CRM livrés par Aurélien, ces travaux ont été réunis dans la branche `feat/dashboard-15` (résolution de quatre conflits mineurs sur des fichiers de réglages partagés). Cette fusion a montré que la solution temporaire `DashboardAuthGate` était devenue inutile : elle connectait le tableau de bord côté navigateur, séparément de la vraie connexion posée par la page `/connexion` (figure 3.1).

L'architecture a donc été changée : le client Supabase du navigateur est passé de `createClient` à `createBrowserClient` (outil `@supabase/ssr`, voir le code en annexe B), ce qui lui permet de **partager les mêmes cookies** que la connexion ouverte par `/connexion`, sans code intermédiaire en plus. Le compte de service, le contrôle d'accès dédié et les variables d'environnement liées ont été supprimés. La chaîne complète a été vérifiée : connexion réelle → cookies → pages du tableau de bord accessibles, RLS vérifiée pour autoriser toute personne connectée et active.

3.8.4 Pourquoi certaines tables restaient vides, et ce qui a changé depuis

Une recherche plus poussée a permis de comprendre pourquoi certaines pages du tableau de bord restaient vides, alors que l'accès aux données marchait désormais :

- la table `missions` était remplie soit par la migration Excel de Mounib (pas encore faite à l'époque), soit par le CRM d'Aurélien, qui utilisait encore des fonctions de données simulées au lieu des vraies routes déjà construites côté serveur ;
- `kpi_setting` n'avait, à ce stade, aucun mécanisme d'alimentation continue dans le code ;
- une première alerte sur `kpi_marketing` (dernière donnée datée du 7 août) s'est révélée être une **fausse piste** : la colonne `date` correspond à la date de *publication* du contenu, et non à la date de sa récupération, comme confirmé en comparant avec la vraie date du dernier post Instagram.

Depuis, ces trois points ont évolué grâce au travail du reste de l'équipe : Aurélien a basculé le module missions du CRM vers les vraies routes API du serveur (le flux devis → mission crée maintenant de vraies lignes dans `missions`), Mounib a importé le contenu marketing réel (posts Instagram et leurs statistiques) dans `contenus` et `kpi_marketing`, ce qui a considérablement rafraîchi les données

affichées sur la page KPI marketing, et il a également terminé la migration des fichiers Excel qui alimente désormais `kpi_setting` avec de vraies données. Les quatre tables utilisées par le tableau de bord sont donc aujourd’hui toutes alimentées.

3.9 Intégration avec la base du CRM

Mon travail a été intégré par-dessus la base commune construite par Aurélien (tâches C1 et C2, déjà fusionnées). Cette intégration a demandé de résoudre des conflits de fichiers : la mise en page partagée `/admin/layout.tsx` d’Aurélien est gardée (elle porte le menu de tout l’espace interne), tandis que la partie propre au tableau de bord (menu latéral, contrôle de connexion) a été déplacée dans une mise en page imbriquée propre à `/admin/dashboard`.

Plus tard dans le stage, les livraisons récentes de Mounib (vraie connexion, migration de la base) et d’Aurélien (modules Devis, Missions, Prospects du CRM) ont été réunies dans la branche `feat/dashboard-15`, ce qui a demandé de résoudre quatre conflits mineurs sur des fichiers de réglages partagés (voir section 3.8 pour le détail du changement de connexion que cette fusion a déclenché). Le tout a été validé et envoyé sur GitHub, sur une branche destinée à une *Pull Request*, comme le demandent les règles du dépôt (fusion relue avant d’être acceptée).

3.10 Refonte du design

Une première version, très animée (effets au survol, dégradés), a été retravaillée vers un style plus sobre : suppression des animations, cartes à bordure fine, couleurs neutres et usage ponctuel des couleurs de l’entreprise (bleu #0063A4, or #D3AF37).

3.11 Démarche de vérification

Chaque nouvelle fonctionnalité a été vérifiée de plusieurs façons :

- vérification des types avec le compilateur TypeScript (`tsc -noEmit`);
- **compilation complète du site** (`next build`) après avoir intégré le code de l’équipe, pour confirmer que tout compile (toutes les routes CRM et tableau de bord);
- test de l’appel réel à OpenRouteService avec la vraie clé;
- test direct de l’API Supabase (connexion, RLS, comptage, ajout/lecture/suppression) pour vérifier la chaîne de données indépendamment de l’interface;
- pour le chatbot, conversations de test menées jusqu’au bout et vérifiées directement en base, plutôt que de faire confiance au seul texte affiché à l’écran (voir section 3.7).

3.12 Difficultés rencontrées

3.12.1 Un cache de compilation abîmé

Le serveur local a arrêté de démarrer (manque de mémoire, puis erreur interne de Turbopack). La cause n’était pas le code mais un dossier de cache (`.next`) abîmé ; le supprimer a résolu le problème — un bon exemple de l’importance de lire les journaux (logs) avant de conclure trop vite à un bug.

3.12.2 Des droits de base non ouverts au départ

Décrit à la section 3.8 : plutôt que de chercher une solution côté tableau de bord, la démarche a été de trouver la vraie cause précisément (test direct de l'API) puis de faire une demande claire à la bonne personne.

3.12.3 Dépendances entre les lots

Une grande partie du lot L5 dépend de livrables d'autres stagiaires (API de Mounib, GPS d'Israa, KPI de David). Le départ d'Israa en cours de stage a confirmé l'intérêt de cette approche : la partie GPS n'a finalement jamais été livrée, sans pour autant bloquer le reste du tableau de bord. La leçon principale a été de construire le tableau de bord pour qu'il reste **toujours présentable**, quel que soit l'état des autres lots (repli en polling, tolérance aux tables vides).

3.12.4 Une fausse alerte sur les données marketing

Une alerte a d'abord été levée : la dernière donnée de `kpi_marketing` semblait dater du 7 août, ce qui pouvait faire penser à une collecte arrêtée. En comparant avec la vraie date du dernier post Instagram, il s'est avéré que la colonne `date` de la table correspond à la date de *publication* du contenu, et non à celle de sa récupération : la donnée était bien à jour, et l'alerte était une fausse piste. Cet épisode montre qu'il faut toujours vérifier le sens exact d'une colonne avant d'en tirer une conclusion.

3.12.5 Du texte généré sans accents

Le texte de préparation du chatbot avait été rédigé sans accents français (probablement par habitude de travailler sur du code, où les accents sont évités). Le modèle de langage reproduisait fidèlement cette orthographe dans ses réponses aux visiteurs, ce qui donnait une impression peu professionnelle sur un outil destiné au public. La correction a été simple une fois la cause identifiée : réécrire le texte de préparation avec les accents corrects. Cet incident rappelle qu'un contenu destiné à être lu par de vrais visiteurs mérite la même rigueur de relecture que le reste du site, même quand il s'agit « seulement » d'un texte de configuration.

Chapitre 4

Conclusion

Le lot L5 est aujourd’hui terminé. Les trois pages du tableau de bord (pilotage du jour, KPI commerciaux, KPI marketing) sont pleinement fonctionnelles et connectées aux vraies données de l’entreprise : les alertes de pilotage remontent les leads chauds et les missions en retard, et les indicateurs commerciaux et marketing sont calculés sur de vraies données avec sélection de période. La migration vers l’authentification réelle (`createBrowserClient`) a été menée à bien avec l’équipe, et `kpi_setting`, dernière table qui restait sans alimentation, est désormais elle aussi renseignée grâce à la migration des fichiers Excel réalisée par Mounib : les trois tables utilisées par le tableau de bord sont donc toutes alimentées avec de vraies données.

En plus du lot L5, l’assistant de discussion proposé par mon maître de stage a également été livré : il répond aux questions des visiteurs du site public et enregistre les prospects intéressés directement dans la base, au même titre que les autres sources du CRM.

La dernière semaine du stage a été consacrée à des tests de bout en bout sur l’ensemble de ces outils, suivis d’une démonstration au gérant de RT MOVE pour lui expliquer comment les utiliser au quotidien.

Ce stage m’a permis de travailler sur un vrai projet d’équipe, avec des dépendances fortes entre les lots des cinq stagiaires (et un imprévu, le départ d’Israa en cours de route). Il m’a surtout appris à construire des outils **robustes face à l’incertitude** : tolérer des données manquantes ou partielles, prévoir des replis automatiques quand une dépendance externe n’est pas disponible, et toujours vérifier directement en base plutôt que de faire confiance au seul rendu à l’écran.

Annexe A

DD&RS — Développement durable et responsabilité sociétale

Le projet contribue à plusieurs objectifs de responsabilité :

- **Moins de papier et de double saisie** : le tableau de bord et le CRM remplacent des fichiers Excel remplis à la main, ce qui limite les erreurs et les impressions.
- **Trajets optimisés** : le calcul d'itinéraires (ORS) et le suivi en direct aident à limiter les kilomètres à vide, donc la consommation de carburant et les émissions.
- **Outils libres** : l'utilisation d'OpenStreetMap et de logiciels open source réduit la dépendance à des services privés.

Annexe B

Détails techniques

B.1 Outil de suivi en direct avec repli polling

```
1 const channel = supabase
2   .channel("positions-changes")
3   .on("postgres_changes",
4     { event: "*", schema: "public", table: "positions_vehicules" },
5     () => fetchVehiclePositions())
6   .subscribe((status) => {
7     if (status === "SUBSCRIBED") {
8       setConnectionStatus("realtime"); stopPolling();
9     } else if (["CHANNEL_ERROR", "TIMED_OUT", "CLOSED"].includes(status)) {
10      setConnectionStatus("polling"); startPolling();
11    }
12  });
13 // Repli si le canal n'atteint jamais SUBSCRIBED
14 fallbackTimer = setTimeout(() => { startPolling(); }, 8000);
```

Listing B.1 – Extrait simplifié – lib/hooks/use-supabase.ts

B.2 Alertes de pilotage

```
1 const today = new Date().toISOString().split("T")[0];
2 // Leads chauds : prospects crees aujourd'hui
3 const { data: prospects } = await supabase
4   .from("prospects").select("id, nom, prenom, type, source, date_creation")
5   .eq("date_creation", today).order("date_creation", { ascending: false });
6 // Missions en retard : date passee, statut non cloture
7 const { data: missions } = await supabase
8   .from("missions").select("id, date_operation, statut")
9   .lt("date_operation", today)
10  .in("statut", ["a_planifier", "planifiee", "en_cours"]);
```

Listing B.2 – Extrait simplifié – lib/hooks/use-alertes.ts

B.3 Appel sécurisé à OpenRouteService (côté serveur)

```
1 const key = process.env.ORS_API_KEY; // cle jamais exposee au client
2 const r = await fetch(ORS_URL, { method: "POST", headers, body });
3 if (!r.ok) {
4   return NextResponse.json({ error: `ORS ${r.status}` },
5     { status: r.status === 429 ? 429 : 502 }); // quota vs indisponible
```

6 }

Listing B.3 – Extrait simplifié – app/api/directions/route.ts

B.4 Client Supabase côté navigateur (vraie connexion)

```

1 import { createBrowserClient } from "@supabase/ssr";
2
3 export function createClient() {
4   return createBrowserClient(
5     process.env.NEXT_PUBLIC_SUPABASE_URL,
6     process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY
7   );
8 }
9 // Partage les cookies de session poses par /connexion :
10 // plus besoin de compte de service ni de garde d'authentification dédiée.

```

Listing B.4 – Extrait simplifié – lib/supabase/client.ts

B.5 Assistant de discussion : réponse en streaming avec appel de l'outil `save_lead`

```

1 // Un seul outil, pas besoin d'un agent complet : boucle manuelle sur 3 tours max.
2 const chunks = await ai.models.generateContentStream({
3   model: "gemini-flash-lite-latest",
4   contents,
5   config: {
6     systemInstruction: CHATBOT_SYSTEM_PROMPT, // FAQ + tarifs reels du site
7     tools: [{ functionDeclarations: [SAVE_LEAD_TOOL] }],
8   },
9 });
10
11 for await (const chunk of chunks) {
12   if (chunk.text) controller.enqueue(encodeEvent({ type: "text", value: chunk.text }));
13   if (chunk.functionCalls?.length) functionCalls.push(...chunk.functionCalls);
14 }
15
16 for (const call of functionCalls) {
17   if (call.name === "save_lead") {
18     // Ecrit ou met à jour la vraie table prospects, avec controle anti-doublon
19     // (email puis telephone) avant insertion.
20     const result = await createLeadFromChatbot(call.args);
21   }
22 }

```

Listing B.5 – Extrait simplifié – app/api/chatbot/route.ts

Bibliographie

- [1] Documentation Next.js. <https://nextjs.org/docs>
- [2] Documentation React. <https://react.dev>
- [3] Documentation Tailwind CSS. <https://tailwindcss.com/docs>
- [4] shadcn/ui. <https://ui.shadcn.com>
- [5] Leaflet.js. <https://leafletjs.com>
- [6] OpenStreetMap. <https://www.openstreetmap.org>
- [7] OpenRouteService. <https://openrouteservice.org>
- [8] Recharts. <https://recharts.org>
- [9] Documentation Supabase (client JS, Realtime, RLS). <https://supabase.com/docs>
- [10] Documentation de l'API Gemini (Google). <https://ai.google.dev/gemini-api/docs>